Top 100 AI Interview Questions & Answers

Senior Lead AI & Data Innovation Engineer

Contents

GenAI & LLM Fundamentals (Q1-Q12)
Prompt Engineering (Q13-Q20)
RAG (Retrieval-Augmented Generation) (Q21-Q38)
Vector Databases, Knowledge Graphs & Semantic Layers (Q39-Q48)
AI Agents & Agentic AI (Q49-Q60)
AI Frameworks (Q61-Q68)
Evaluation, Guardrails & Observability (LLMOps) (Q69-Q80)
AI on Snowflake (Cortex, Snowpark, Native Apps, Openflow) (Q81-Q88)
Cloud AI Platforms & Data Platforms (AWS-first) (Q89-Q94)
Innovation, POCs & Thought Leadership (Q95-Q100)

GenAI & LLM Fundamentals

Q1. Explain how transformer-based LLMs work at a high level (tokens, attention, context window).

Text is split into **tokens**, each mapped to an embedding vector. Stacked transformer layers use **self-attention** so every token can weigh every other token in the context, building contextual representations. The model predicts the next-token probability distribution and generates text one token at a time. The **context window** is the maximum number of tokens (prompt plus output) the model can attend to in one call.

Q2. What is the difference between pre-training, fine-tuning, instruction tuning and RLHF/RLAIF?

- **Pre-training:** next-token prediction on huge corpora; gives general language and world knowledge.
- **Fine-tuning:** further training on task- or domain-specific data (full or parameter-efficient such as LoRA).
- **Instruction tuning:** fine-tuning on instruction-response pairs so the model follows directions.
- **RLHF / RLAIF:** aligning behavior using preference signals from humans (RLHF) or from an AI judge (RLAIF), usually via a reward model or direct preference optimization.

Q3. How do temperature, top-p, top-k and max tokens affect output?

Temperature rescales the token distribution: low values give focused, repeatable output; high values give more varied output. **Top-p** samples from the smallest set of tokens whose cumulative probability exceeds p; **top-k** samples from the k most likely tokens. **Max tokens** caps output length (and cost). For extraction, SQL and classification use low temperature; for brainstorming use higher.

Q4. What causes hallucinations, and how do you reduce them in production?

LLMs generate plausible text, not verified facts; gaps in knowledge, ambiguous prompts, stale data and missing context all cause fabrication. Mitigations: ground answers with RAG, require citations, instruct the model to say "I don't know", constrain output with schemas, lower temperature, add verification or self-check steps, use evaluation sets that measure faithfulness, and keep a human in the loop for high-risk decisions.

Q5. Compare open-source and proprietary models, and give your criteria for choosing between them.

Proprietary models (API-based) typically lead on capability and are fast to adopt, but involve data-governance review, per-token cost and vendor dependence. **Open-weight** models offer control, customization, private deployment and predictable cost at scale, but need infrastructure and MLOps effort. Criteria: task quality on your own evals, data sensitivity and residency, latency, cost at expected volume, licensing terms, and team skills.

Q6. Compare small language models and large models. When is the smaller one the right choice?

Small models are cheaper, faster, easier to host privately and fine-tune, and are often enough for narrow, well-defined tasks (classification, extraction, routing, summarization). Large models handle open-ended reasoning, broad knowledge and complex instructions. Common pattern: **route** simple requests to a small model and escalate hard ones, validated by evals on your own data.

Q7. What are embeddings, and how do embedding models differ from generative models?

Embeddings are dense numeric vectors that capture semantic meaning, so similar texts (or images) land close together in vector space. **Embedding models** output a vector for an input and are used for search, clustering, deduplication and recommendation; **generative models** produce new tokens. Both are needed in RAG: embeddings to retrieve, a generative model to answer.

Q8. Explain tokenization and why it matters for cost, latency and multilingual text.

Tokenizers split text into subword units (BPE-style). Providers bill and limit by tokens, so token count drives cost, latency and context usage. Different languages, code and numbers tokenize at different efficiency (many non-English languages need more tokens per word), which affects cost and effective context size. Measure with the model's own tokenizer, not word counts.

Q9. What are context window limits, and how do you handle documents that exceed them?

Options: **chunk and retrieve** only relevant parts (RAG); **map-reduce summarization** (summarize chunks, then summarize summaries); **hierarchical or iterative refinement**; compress or prune context; and use long-context models selectively. Be aware that very long contexts raise cost and latency, and accuracy can degrade for details buried in the middle.

Q10. Explain multimodal models and a business use case for them.

Multimodal models accept and sometimes produce several modalities (text, images, audio, video, documents). Use case: **document intelligence** - extracting fields from invoices, forms and scanned contracts with layout understanding, instead of brittle OCR rules; or reading charts and screenshots in support workflows. Evaluate on your own documents, since layout-heavy and handwritten inputs vary.

Q11. What is structured output / JSON mode / function calling, and why is it important for enterprise apps?

These features constrain the model to emit output matching a schema (JSON schema, tool signature), or to request a function call with typed arguments. Downstream code can then parse results reliably, validate them (for example with Pydantic), and trigger actions safely. This removes fragile regex parsing and is the foundation for tool use and agents.

Q12. How do reasoning models differ from standard LLMs, and when are they worth the extra cost and latency?

Reasoning models spend additional inference-time compute on an internal chain of thought before answering, improving multi-step problems (math, planning, complex code, ambiguous analysis). They cost more and respond slower. Use them for hard, high-value tasks; use faster standard models for extraction, classification, chat and simple Q&A. Many systems route between the two.

Prompt Engineering

Q13. Compare zero-shot, few-shot, chain-of-thought and ReAct prompting.

- **Zero-shot:** instruction only.
- **Few-shot:** include examples of the desired input-output behavior, which improves format and style consistency.
- **Chain-of-thought:** ask the model to reason step by step (or use a reasoning model) for multi-step problems.
- **ReAct:** interleaves reasoning with tool actions and observations; the basis of many agents.

Q14. How do you design a robust system prompt for an enterprise assistant?

Define role and scope, tone, allowed and disallowed topics, the knowledge sources to rely on, an output format, how to cite sources, how to handle uncertainty ("say you don't know"), escalation rules, and safety rules. Keep it structured (sections or XML-style tags), put critical rules clearly, test with adversarial inputs, and version it in source control.

Q15. How do you version, test and manage prompts like code?

Store prompts in Git (or a prompt registry), parameterize with templates, tag versions, and attach an evaluation set. Run automated evals on every prompt change in CI, compare against the baseline, and gate releases on thresholds. Log which prompt version produced each production trace so regressions can be traced.

Q16. What is prompt injection (direct and indirect), and how do you mitigate it?

Direct: a user tries to override instructions. **Indirect:** malicious instructions are hidden in retrieved documents, web pages or tool output. Mitigations: treat all external content as untrusted data, separate instructions from data, least-privilege tools, output validation, input/output guardrails, human approval for sensitive actions, sandboxing, and monitoring. There is no single complete fix, so use defense in depth.

Q17. How do you make prompts produce consistent, parseable output?

Use structured output or function-calling modes with a schema, low temperature, explicit format instructions with examples, validation (Pydantic/JSON schema), and automatic retry or repair when validation fails. Keep fields simple and enumerate allowed values.

Q18. How do you reduce prompt token cost without hurting quality?

Trim redundant instructions, retrieve fewer but better chunks (re-ranking), compress or summarize history, use prompt caching for static prefixes, route to smaller models, cap output length, batch requests where latency allows, and measure quality on an eval set after each change.

Q19. What is prompt caching, and when does it pay off?

Providers can cache the processed static prefix of a prompt (system prompt, long documents, tool definitions) so repeated calls pay reduced cost and latency for that prefix. It pays off when many requests share a large identical prefix, so put static content first and variable content last.

Q20. How do you approach prompt optimization systematically?

Define a metric and a representative eval set first. Then iterate: error analysis on failures, targeted prompt changes, few-shot example selection, and automated optimization (DSPy-style compilers or search-based methods) that tune prompts against the metric. Always validate on a held-out set to avoid overfitting.

RAG (Retrieval-Augmented Generation)

Q21. Explain a RAG architecture end to end.

Offline: ingest documents, parse and clean, chunk, create embeddings, store in a vector (or hybrid) index with metadata. **Online:** take the user query (optionally rewrite it), retrieve candidates (vector plus keyword), re-rank, filter by permissions, assemble a prompt with the top chunks, generate an answer with citations, then apply output guardrails and log traces for evaluation.

Q22. When would you choose RAG over fine-tuning, and when the reverse?

RAG for knowledge that changes, large private corpora, citations and access control. **Fine-tuning** for consistent style/format, domain-specific behavior, narrow task accuracy or lower latency with smaller models. They are complementary: fine-tune for behavior, retrieve for facts.

Q23. Compare chunking strategies.

- **Fixed-size:** simple, but may cut through ideas.
- **Recursive:** splits on paragraph, sentence and word boundaries; a strong default.
- **Semantic:** splits where meaning shifts, using embeddings.
- **Structure-aware:** respects headings, tables and sections (best for manuals, contracts).
- **Parent-child:** retrieve small chunks for precision but pass the larger parent for context.

Q24. How do you choose chunk size and overlap?

Start with a few hundred tokens and modest overlap (about 10-20 percent), then tune empirically using retrieval metrics on your eval set. Smaller chunks improve precision but lose context; larger chunks keep context but dilute relevance and raise cost. Content type matters: FAQs need small chunks, legal text needs section-aware chunks.

Q25. How do you select an embedding model for a domain-specific corpus?

Build a labeled retrieval test set from real queries, then compare candidate models on recall@k and MRR. Consider dimension size (storage and speed), max input length, multilingual support, domain fit, latency, cost and hosting constraints. Public benchmarks are a starting point only.

Q26. Explain hybrid search and why it often beats vector-only search.

Hybrid combines **keyword search** (BM25; exact terms, IDs, product codes, acronyms) with **vector search** (semantic similarity), merging ranked lists with methods like reciprocal rank fusion. Vector-only search can miss exact identifiers, while keyword-only misses paraphrases, so the combination is more robust.

Q27. What is re-ranking, and where does it fit in the pipeline?

A re-ranker (cross-encoder or LLM) rescoring the top N retrieved candidates against the query for finer relevance. Pipeline: retrieve broadly (say 50), re-rank, pass the top few (say 5) to the LLM. It improves precision and lets you use fewer context tokens.

Q28. Explain query rewriting, query expansion and HyDE.

Rewriting clarifies or de-contextualizes a query (resolving pronouns from chat history). **Expansion** generates multiple variants or sub-queries to improve recall. **HyDE** has the LLM write a hypothetical answer, then embeds that text to retrieve similar real documents. They help with vague or short queries but add latency and cost.

Q29. What is multi-hop retrieval, and how do you support it?

Multi-hop questions need information from several documents combined (for example, "Which supplier of component X had the most delayed orders?"). Approaches: query decomposition into sub-questions, iterative retrieve-reason loops (agentic RAG), knowledge graphs, and entity links between chunks.

Q30. How do you handle tables, PDFs, scanned documents and images in a RAG pipeline?

Use layout-aware parsing (document AI or parsing services) and OCR for scans; preserve tables as structured text (markdown or JSON) with captions; extract images with captions or multimodal descriptions; keep page and section metadata. Validate parsing quality with spot checks, because poor parsing is a leading cause of poor RAG answers.

Q31. How do you apply metadata filtering and access control in RAG?

Store attributes (department, region, classification, owner, ACL groups) with each chunk and filter at retrieval time using the caller's identity, never after generation. Prefer platform-native row-level security or filtered search so users cannot retrieve documents they cannot access in the source system, and log access for audit.

Q32. What is GraphRAG, and when does it outperform vector RAG?

GraphRAG extracts entities and relationships into a knowledge graph and retrieves via graph traversal and community summaries. It outperforms vector RAG for relationship-heavy, multi-hop and corpus-wide "global" questions (themes, connections across documents), at higher build cost and complexity.

Q33. How do you keep a RAG index fresh?

Use incremental ingestion with change detection (hashes, modified timestamps, CDC), upsert and delete chunks by document ID, version embeddings when you change the embedding model (re-index fully), schedule refreshes by content volatility, and monitor index lag.

Q34. How do you evaluate retrieval quality?

Create a labeled set of queries with relevant documents/chunks and measure **recall@k**, **precision@k**, **MRR** and **nDCG**. RAG-specific metrics include **context precision** and **context recall**. Evaluate retrieval separately from generation so you know which component to fix.

Q35. How do you evaluate generation quality?

Measure **faithfulness/groundedness** (claims supported by the context), **answer relevance**, completeness, correctness against reference answers, and citation accuracy. Combine automated LLM-as-judge scoring with periodic human review and production feedback.

Q36. A RAG chatbot gives wrong answers. Walk through how you'd debug retrieval vs. generation.

1. Pull the trace: query, rewritten query, retrieved chunks, final prompt, answer.
2. If the right chunk was **not retrieved**: check parsing, chunking, embeddings, filters, hybrid weights, and add re-ranking or query rewriting.
3. If it **was retrieved but ignored or misused**: tighten the prompt, reduce noise, reorder context, use a stronger model.
4. If the knowledge is **absent**: fix the content gap.

5. Add the failure to the eval set so it cannot regress.

Q37. How do you add citations and source attribution to answers?

Keep document and chunk IDs with every retrieved passage, instruct the model to reference them, return structured output with claims plus source IDs, and verify that cited passages actually support the statements. Show clickable source links and page numbers in the UI.

Q38. What is agentic RAG, and how does it differ from naive RAG?

Naive RAG does one retrieve-then-generate pass. **Agentic RAG** lets an agent decide whether and what to retrieve, choose among tools or sources (vector store, SQL, web), reformulate queries, evaluate whether results are sufficient, and iterate. It handles complex questions better but costs more latency and needs tighter guardrails.

Vector Databases, Knowledge Graphs & Semantic Layers

Q39. Compare vector databases (pgvector, Pinecone, Weaviate, Qdrant, OpenSearch, Snowflake/Databricks vector search).

- **pgvector:** vectors inside PostgreSQL; simple, transactional, good for modest scale.
- **Pinecone:** fully managed, low operational burden.
- **Weaviate / Qdrant:** open-source engines with rich filtering and hybrid search; self-host or managed.
- **OpenSearch:** strong hybrid keyword plus vector search in an existing search stack.
- **Platform-native (Snowflake, Databricks):** vectors live beside governed data, inheriting security and avoiding data movement.

Choose by scale, filtering and hybrid needs, ops capacity, governance and existing stack.

Q40. Explain ANN indexes (HNSW, IVF, PQ) and their trade-offs.

Approximate nearest neighbor indexes trade a little recall for big speedups. **HNSW** is a layered proximity graph: very fast and accurate but memory-hungry. **IVF** clusters vectors and searches only nearby clusters: lighter on memory, needs training and tuning (number of probes). **PQ** compresses vectors into short codes to save memory at some accuracy cost; it is often combined with IVF.

Q41. Which similarity metrics exist, and how do you choose?

Cosine similarity compares direction (most common for text embeddings), **dot product** accounts for magnitude (equivalent to cosine on normalized vectors), and **Euclidean (L2)** measures distance. Use the metric the embedding model was trained for, and keep it consistent between indexing and querying.

Q42. How do you scale a vector store to hundreds of millions of vectors?

Shard and replicate, use compressed or quantized indexes (PQ, scalar quantization), tune HNSW/IVF parameters, store vectors on disk-friendly indexes, reduce dimensions where quality allows, pre-filter on metadata, and separate hot from cold data. Load test recall and latency at target scale.

Q43. When would you use a knowledge graph in an AI solution?

When relationships and multi-hop reasoning matter: supply chains, fraud, compliance, drug or product relationships, enterprise entity resolution. A graph supports explainable answers, consistent entity definitions and complements vector search for global or relational questions.

Q44. How do you build a knowledge graph from unstructured text using LLMs?

Chunk documents, have an LLM extract entities and relations against a defined schema/ontology, normalize and deduplicate entities (entity resolution), load nodes and edges into a graph database (Neo4j, Neptune), link back to source chunks, and validate with sampling. Controlling the schema is key to quality.

Q45. What is a semantic layer, and how does it improve text-to-SQL and BI accuracy?

A semantic layer defines business concepts once: metrics (revenue, churn), dimensions, relationships, synonyms and join logic. Natural-language tools then map questions to governed definitions rather than guessing from raw table and column names, giving consistent and more accurate answers.

Q46. How do semantic models or semantic views reduce ambiguity for natural-language analytics?

They add descriptions, synonyms, verified metric formulas, example queries and relationships on top of physical tables. The LLM sees business vocabulary and approved logic, which cuts wrong joins, wrong aggregations and ambiguous column choices. Snowflake's semantic models/views for Cortex Analyst are an example.

Q47. How would you design a text-to-SQL system and measure its accuracy?

Components: semantic layer or schema context, retrieval of relevant tables and sample queries, SQL generation, validation (parse, dry-run, read-only role), execution, and explanation. Measure **execution accuracy** (result equals expected result) on a golden set of question-SQL pairs, plus latency, cost and failure categories. Improve by adding verified queries and fixing semantic definitions.

Q48. How do you stop text-to-SQL from generating unsafe or expensive queries?

Use a read-only, least-privilege role with row/column policies; allow-list schemas; enforce query timeouts, row limits and cost estimates; parse and block DDL/DML; run on a small dedicated warehouse or resource-monitored compute; and log all generated SQL for audit.

AI Agents & Agentic AI

Q49. What is an AI agent, and how does it differ from a chain or workflow?

A **workflow/chain** follows a predefined path of steps. An **agent** uses an LLM to decide dynamically which tools to call and in what order until the goal is met. Agents suit open-ended tasks; fixed workflows suit predictable processes and are cheaper and more reliable.

Q50. Explain the main agent patterns.

- **ReAct:** reason, act, observe loops.
- **Plan-and-execute:** plan first, then run steps.
- **Reflection:** the agent critiques and revises its own output.
- **Multi-agent:** specialized agents collaborate.
- **Supervisor/router:** a coordinator delegates to specialist agents or tools.

Q51. How does tool/function calling work, and how do you design good tools?

You describe tools (name, description, JSON-schema parameters); the model returns a structured call; your code executes it and returns the result for the model to continue. Good tools are narrow, well named, clearly documented, have validated typed inputs, return concise structured outputs and helpful error messages, and are idempotent where possible.

Q52. What is the Model Context Protocol (MCP), and why does it matter?

MCP is an open protocol that standardizes how AI applications connect to external tools, data sources and prompts through MCP servers (using JSON-RPC). It replaces many custom integrations with one interface, so any MCP-capable client can use any MCP server. Treat servers as a security boundary: authenticate, scope permissions and review tools.

Q53. What is Agent2Agent (A2A) communication, and how does it complement MCP?

A2A is an open protocol for agents built by different vendors or frameworks to discover each other (via agent cards) and delegate tasks. **MCP connects an agent to tools and data; A2A connects agents to other agents.** Together they enable interoperable multi-agent systems.

Q54. How do you manage short-term and long-term agent memory?

Short-term: conversation state and scratchpad within the context window, trimmed or summarized as it grows. **Long-term:** persisted facts, preferences and past outcomes in a database or vector store, retrieved when relevant. Add rules for what to store, expiry, user control and privacy.

Q55. How do you make agents reliable?

Prefer deterministic code for fixed steps, constrain tools and outputs with schemas, add retries and timeouts, validate intermediate results, checkpoint state, use human-in-the-loop approvals for risky actions, and test with scenario-based evals. Reduce autonomy where the process is known.

Q56. How do you prevent runaway loops and uncontrolled cost in agents?

Cap iterations, tokens, tool calls and wall-clock time per task; set budgets per user; detect repeated actions; use cheaper models for sub-steps; cache tool results; and alert on anomalies in traces.

Q57. How do you secure agents?

Least-privilege credentials per tool, allow-listed tools, sandboxed code execution, user-scoped identity pass-through, approval gates for write or destructive actions, input/output guardrails against prompt injection, secrets management, and complete audit logs of tool calls.

Q58. When should you not use an agent?

When the process is deterministic, latency and cost must be tightly bounded, errors are costly, or a simple RAG/workflow solves it. Agents add non-determinism, cost and testing complexity, so start with the simplest architecture that meets the requirement.

Q59. Design a multi-agent system for a business process (for example claims or procurement).

Example (claims): a supervisor agent routes tasks to an **intake agent** (extract fields from documents), a **policy agent** (RAG over policy rules), a **fraud-check agent** (calls scoring service/SQL), and a **drafting agent** (prepares decision letter). Add shared state, human approval for payouts above a threshold, audit trails, guardrails, and evals per agent plus end to end.

Q60. How do you evaluate and test an agent's trajectory, not just its final answer?

Log the full trace (reasoning, tool calls, arguments, results). Score tool selection correctness, argument accuracy, step efficiency, loop count, and recovery from errors, in addition to final-answer quality. Use scenario suites with simulated tools and replay production traces for regression.

AI Frameworks

Q61. Compare LangChain, LangGraph and LlamaIndex, and say when you'd use each.

LangChain: broad toolkit of integrations and components for LLM apps. **LangGraph:** graph/state-machine orchestration with persistence, branching and human-in-the-loop for agents. **LlamaIndex:** strong data ingestion, indexing and retrieval abstractions, ideal for RAG-centric apps. Use LlamaIndex for retrieval-heavy work, LangGraph for stateful agent flows, and LangChain for integrations.

Q62. Why use LangGraph (state machines, checkpoints, human-in-the-loop) over simple chains?

Real agent flows need loops, conditional branches, shared state, resumability after failures, and approval pauses. LangGraph models these explicitly as nodes and edges with checkpointed state, making flows observable, testable and controllable compared with linear chains.

Q63. Compare CrewAI, AutoGen and Semantic Kernel for multi-agent orchestration.

CrewAI: role-based agent teams with simple task definitions; quick to prototype. **AutoGen:** conversational multi-agent patterns and flexible agent interactions. **Semantic Kernel:** Microsoft SDK (C#, Python, Java) with plugins and planners, suited to enterprise .NET and Azure environments. Pick by language/ecosystem and control needs; verify current project status before committing.

Q64. What are the risks of framework lock-in and abstraction overhead, and when would you skip a framework?

Frameworks change fast, hide prompts and behavior behind abstractions, complicate debugging and add dependencies. For simple apps, direct SDK calls with a thin internal wrapper can be clearer and more stable. Use frameworks where they add real value (state management, integrations) and isolate them behind your own interfaces.

Q65. How do you implement streaming responses and async calls in an LLM application?

Use the provider's streaming API and relay tokens to clients via server-sent events or WebSockets; use `async`/`await` with concurrent calls for parallel retrieval and tool use; apply backpressure, timeouts and cancellation. Streaming greatly improves perceived latency.

Q66. How do you build a reusable LLM application scaffold or SDK for delivery teams?

Provide a templated repo with configuration, model/provider abstraction, retrieval modules, prompt management, guardrails, tracing, eval harness, CI, Docker and IaC, plus documentation and sample apps. Version it, publish to an internal package index, gather feedback from delivery teams and keep it opinionated but extensible.

Q67. How do you wrap an LLM app in a production API?

Use FastAPI with typed request/response models, authentication (OAuth/JWT), rate limiting, input validation, response streaming, caching, structured logging and tracing, health checks, and async execution. Handle model timeouts and fallbacks, and keep secrets in a secret manager.

Q68. How do you containerize and deploy an AI service?

Package with Docker (slim base image, pinned dependencies), deploy to Kubernetes (EKS), ECS/Fargate or serverless depending on scale, use autoscaling, GPU nodes only for self-hosted models, IaC (Terraform) and CI/CD with staged environments. Externalize configuration and secrets, and add readiness probes and observability.

Evaluation, Guardrails & Observability (LLMOps)

Q69. How do you build an evaluation dataset?

Combine a **golden set** curated with domain experts, **production traces** sampled and labeled, and **synthetic** question-answer pairs generated from documents (then human-reviewed). Cover happy paths, edge cases, adversarial inputs and known failures. Version the dataset and grow it continuously.

Q70. Explain LLM-as-a-judge: strengths, biases and how to calibrate it.

A strong LLM scores outputs against a rubric, scaling evaluation cheaply. Known biases: position bias, verbosity bias, self-preference, and rubric misreading. Calibrate by comparing judge scores to human labels on a sample, using clear rubrics with examples, randomizing order, using a different judge model, and re-checking periodically.

Q71. Compare evaluation frameworks and tools.

Ragas and **DeepEval:** metrics for RAG and LLM outputs in code. **TruLens:** feedback functions plus tracing. **promptfoo:** prompt/model comparison and CI-friendly testing. **LangSmith / Langfuse:** tracing, datasets and evals. **MLflow evaluation:** fits teams already on MLflow. Pick by stack, CI integration and metric flexibility.

Q72. How do you run regression tests for prompts and models in CI/CD?

Keep a fixed eval set in the repo, run it on every prompt, model or retrieval change, compare metrics with the main-branch baseline, and fail the build when scores drop beyond a threshold. Use deterministic settings (low temperature, seeds where possible) and record cost and latency too.

Q73. What guardrails would you add on input and output?

Input: PII detection/redaction, prompt-injection and jailbreak detection, topic restrictions, length limits. Output: toxicity and harmful-content filters, PII leakage checks, schema validation, groundedness checks, and policy compliance. Add fallbacks (safe refusals) and log all blocked events.

Q74. Compare guardrail options (Bedrock Guardrails, NeMo Guardrails, Guardrails AI, Cortex Guard).

Bedrock Guardrails: managed content, topic, PII and grounding controls inside AWS. **NeMo Guardrails:** open-source programmable dialogue rails. **Guardrails AI:** open-source validators for structure and content. **Cortex Guard:** Snowflake's safety filter for Cortex LLM calls. Choose based on where your models run and how much customization you need.

Q75. What should you trace and log in an LLM application?

Request ID, user (pseudonymized), prompt version, model, parameters, retrieved chunks, tool calls and results, token counts, latency per step, cost, final output, guardrail events and user feedback. Mind privacy: mask sensitive data and apply retention policies.

Q76. How do you monitor drift, quality decay and cost in production?

Track online metrics (feedback rates, escalation rate, groundedness scores on sampled traffic, refusal rate), embedding or query-distribution drift, retrieval hit quality, latency and token spend per feature. Set alerts, review samples weekly and re-run offline evals when data or models change.

Q77. How do you collect and use human feedback?

Capture thumbs up/down with reasons, corrections and escalations in the UI; route low-rated traces to a review queue; convert reviewed failures into eval cases and few-shot examples; and use accumulated preference data for tuning where justified.

Q78. How do you handle model version upgrades or deprecations without breaking applications?

Abstract the model behind configuration, pin versions, track provider deprecation notices, run the full eval suite on the candidate, shadow-test or canary a share of traffic, compare quality/cost/latency, and keep rollback ready. Re-tune prompts if behavior shifts.

Q79. How do you control LLM cost?

Model routing (small model first), caching (semantic and prompt caching), batching offline workloads, shorter prompts and outputs, better retrieval (fewer chunks), per-user and per-feature token budgets, and cost dashboards. Reserve expensive reasoning models for high-value paths.

Q80. What are the main LLMOps differences from classic MLOps?

Artifacts are prompts, retrieval indexes, tool definitions and guardrails rather than only trained weights. Evaluation is open-ended and often needs LLM judges and human review. Costs scale with tokens, behavior changes with provider model updates, and new risks arise (prompt injection, hallucination). Tracing and continuous evaluation become central.

AI on Snowflake (Cortex, Snowpark, Native Apps, Openflow)

Q81. What does Snowflake Cortex offer, and how do the parts fit together?

Cortex provides AI inside Snowflake's governance boundary: **LLM/AISQL functions** (complete, summarize, classify, extract, translate, etc.), **Cortex Search** (hybrid vector and keyword search over text), **Cortex Analyst** (natural-language questions over structured data via semantic models), **Cortex Agents** that orchestrate search and analyst tools, and Snowflake Intelligence as a business-user interface. Features evolve quickly, so check current docs.

Q82. How would you build a RAG app with Cortex Search and Cortex LLM functions?

Load documents to a stage, parse them (document parsing functions), chunk text into a table, create a **Cortex Search service** on the chunk column with attributes for filtering, query the service for top results, pass the chunks to a Cortex LLM function (such as `COMPLETE`) with a grounding prompt, and surface it through Streamlit in Snowflake or an API. Secure access with roles and row policies.

Q83. How does Cortex Analyst turn natural language into SQL, and what role do semantic models play?

Cortex Analyst takes a question plus a **semantic model** (YAML or semantic view) describing tables, dimensions, measures, synonyms, relationships and verified queries, uses LLMs to generate SQL grounded in that model, and returns SQL and results. A good semantic model is the main lever for accuracy.

Q84. How would you use Snowpark for feature engineering and ML, and how does Snowflake ML compare to external training?

Snowpark lets you write DataFrame-style Python that executes inside Snowflake, so feature engineering and scoring happen next to governed data with no data export. Snowflake ML adds modeling APIs, a Model Registry, Feature Store and distributed training on container runtime. External platforms (SageMaker, Databricks) may offer broader frameworks and GPU flexibility; choose based on data gravity, governance and model needs.

Q85. How do Native Apps work, and how would you package an AI accelerator as one?

A Native App is packaged code and data (setup script, manifest, Streamlit UI, procedures, UDFs) in an application package, distributed through listings and installed in the consumer's account, where it runs against their data without that data leaving. To package an AI accelerator: bundle the pipeline, Cortex calls and UI, define required privileges in the manifest, version it, test installation, and publish privately or on the Marketplace.

Q86. What is Openflow, and how would you use it to ingest unstructured data for AI?

Openflow is Snowflake's managed, NiFi-based data integration service with connectors for databases, SaaS apps, streaming and unstructured sources (document stores such as SharePoint or Drive). Use it to pull documents, land them in stages/tables, then trigger parsing, chunking and Cortex Search indexing, keeping lineage and governance in Snowflake.

Q87. How do Iceberg tables support open, multi-engine AI and analytics architectures?

Apache Iceberg is an open table format; Snowflake can read and write Iceberg tables (Snowflake- or externally-managed catalog), so Spark, Trino, Databricks or Flink can share the same data. This avoids duplicate copies and lock-in while allowing AI workloads in different engines to use common data. Mind catalog choice and governance across engines.

Q88. When would you use Snowpark Container Services for GPU or custom model workloads?

When you need custom containers, open-source model hosting, GPU inference or training, or long-running services that Cortex functions do not cover, while keeping data and security inside Snowflake. Use compute pools (CPU/GPU), image repositories and services; compare cost and operations effort with managed Cortex models.

Cloud AI Platforms & Data Platforms (AWS-first)

Q89. Compare Amazon Bedrock, SageMaker and a self-hosted model on EC2/EKS for GenAI workloads.

Bedrock: managed access to many foundation models via one API, with Knowledge Bases, Guardrails and Agents; fastest path with least ops. **SageMaker:** managed training, tuning and hosting of your own or open models with more control. **EC2/EKS self-hosting:** maximum control and potentially lowest unit cost at scale (vLLM and similar), but highest operational burden.

Q90. How would you build a Bedrock-based RAG solution?

Store documents in S3, create a **Knowledge Base** (chunking, embeddings, vector store such as OpenSearch Serverless or Aurora pgvector), expose retrieval through `RetrieveAndGenerate` or the retrieve API, apply **Guardrails**, optionally add **Agents** for tool use, secure with IAM and VPC endpoints, and monitor with CloudWatch plus evaluation jobs.

Q91. Compare Snowflake Cortex, Databricks Mosaic AI and Bedrock for enterprise GenAI.

Cortex: AI next to Snowflake data with SQL-first access, minimal data movement and governance inherited. **Databricks Mosaic AI:** Lakehouse-centric stack (model serving, vector search, agent framework, MLflow evaluation) with strong custom ML/LLM capability. **Bedrock:** broad model choice and AWS-native integration, independent of any data platform. Choose by where data lives, team skills and governance needs; many enterprises combine them.

Q92. How do you keep enterprise data private when calling LLMs?

Use enterprise offerings with contractual no-training guarantees, private connectivity (VPC endpoints/PrivateLink), in-region processing and data residency controls, encryption, PII masking before prompts, least-privilege access, audit logging, and retention limits. Prefer in-platform AI (Cortex, Bedrock in your account) so data does not leave governed boundaries.

Q93. How do you design an AI-ready data platform?

Foundations: reliable ingestion, data quality checks, lineage, catalog and metadata, governance and access control, support for unstructured data (parsing and storage), embeddings/vector capability, a semantic layer for metrics, feature management, and observability. Treat documents and conversation logs as governed data assets.

Q94. How would you generate synthetic data for testing and evaluation, and what are its limits?

Use LLMs or rule-based/statistical generators (for tabular data, for example Faker or SDV) to create realistic records, questions and edge cases, with schema constraints and seeding for reproducibility. Limits: may miss real-world messiness and biases, can leak patterns from source data, and needs human review and comparison with real distributions.

Innovation, POCs & Thought Leadership

Q95. How do you take an idea from concept to working POC within days?

Define a narrow, demonstrable question and success criteria; use managed services and existing SDKs rather than building infrastructure; start with a thin end-to-end slice (data to model to UI such as Streamlit); use a small realistic dataset; timebox; demo early for feedback; and document assumptions and shortcuts so the next step is clear.

Q96. How do you decide whether a POC is worth productizing, and which success metrics do you use?

Evaluate business value (time or cost saved, revenue impact), technical feasibility, quality against an eval set, unit economics (cost per request), risk and compliance, data readiness and adoption likelihood. Set explicit thresholds up front and make go, iterate or stop decisions on evidence.

Q97. How do you turn a one-off POC into a reusable accelerator adopted by delivery teams?

Generalize the common parts into modules or a template repo, parameterize configuration, add tests, documentation, a quick-start and a demo, publish internally with versioning and a changelog, run enablement workshops, collect feedback from first projects and assign ownership for maintenance.

Q98. How do you stay current with fast-moving AI tooling, and how do you separate hype from value?

Follow release notes, research papers, vendor blogs and community projects; allocate regular hands-on experimentation time; keep a lightweight evaluation template (task, baseline, metric, cost) to test new tools on realistic problems; and favor evidence from your own benchmarks over marketing. Share findings through short internal write-ups.

Q99. How do you explain an AI architecture to executives vs. engineers, and structure a workshop or whitepaper?

Executives: business problem, outcomes, risk, cost and timeline using simple diagrams. **Engineers:** components, data flows, trade-offs, code and evaluation results. A workshop: objectives, short concept session, guided hands-on lab, common pitfalls, Q&A and takeaways. A whitepaper: problem, approach, architecture, results and metrics, lessons learned, recommendations.

Q100. Walk through a recent POC or demo you built: the problem, the design choices, the evaluation, and what you'd change.

Use this structure for your own story (STAR-style, 2-3 minutes):

1. **Problem and users:** what business pain, who benefits, success metric.
2. **Design choices:** data sources, models, RAG/agent pattern, platform (for example Snowflake Cortex or Bedrock) and why alternatives were rejected.
3. **Build:** how you reached a working prototype fast; frameworks and tooling used.
4. **Evaluation:** eval set, metrics (faithfulness, accuracy, latency, cost), results and failure analysis.
5. **Outcome and reuse:** demo feedback, accelerator produced, adoption.
6. **Retrospective:** what you would change (better chunking, semantic model, guardrails, observability, cost controls).

Prepare one concrete example with real numbers; interviewers probe depth and honesty.